

UPSSITECH, Université de Toulouse
Niveau : SRI 4A S8
Module : Stage 4A
Date : 08/06/2026 - 31/08/2026
Lieu : LAAS-CNRS, Toulouse



Rapport de stage

Migration ROS2 et évolution du TER Atelier Flexible

Rédigé par :

Samy Janati

Encadré par :

Sandra Ulrich Ngueveu

Résumé

Ce rapport présente les travaux réalisés durant mon stage de fin de 4ème année au LAAS-CNRS, au sein de l'équipe ROC, sur le TER « Atelier Flexible » de l'ENSEEIH. Ce module pédagogique permet aux étudiants de commander une cellule de production automatisée simulée sous CoppeliaSim en écrivant un réseau de Petri, communiquant avec la simulation via ROS. Le TER a récemment été migré de ROS 1 vers ROS 2 par un binôme d'étudiants, une migration rendue nécessaire par l'obsolescence de ROS 1 et le passage des ordinateurs de l'école sous Ubuntu 24.04.

Mon stage s'est articulé autour de deux objectifs. Le premier a consisté à vérifier que cette migration était fonctionnelle, en soumettant la simulation à des scénarios de production complets plutôt qu'à des simples vérifications de démarrage. Cette démarche m'a permis d'identifier que les durées de fabrication codées dans les fichiers de configuration devaient être réajustées, la simulation ROS 2 s'exécutant plus rapidement que son équivalent ROS 1.

Le second objectif, qui constitue la contribution principale de ce stage, a consisté à faire évoluer la simulation en permettant l'évacuation d'un produit depuis n'importe quel poste de la cellule, alors que seul le poste 3 le permettait jusqu'ici. Cette évolution a nécessité de comprendre puis de modifier l'ensemble de la chaîne logicielle concernée : l'interface de commande utilisée par l'étudiant, la réception et le traitement de la demande d'évacuation côté robot, l'enregistrement de l'information dans le journal de simulation, et enfin la vérification automatique de la conformité par l'outil de correction du TER. Chaque brique a été testée isolément avant intégration, avec un souci constant de compatibilité avec les codes étudiants existants.

Mots-clés : ROS 2, migration logicielle, réseau de Petri, simulation robotique, atelier flexible, CoppeliaSim

Abstract

This report presents the work carried out during my fourth-year internship at LAAS-CNRS, within the ROC team, on the "Flexible Workshop" educational project at ENSEEIH. This teaching module allows students to control a simulated automated production cell by writing a Petri net that communicates with a CoppeliaSim simulation through ROS. The project was recently migrated from ROS 1 to ROS 2 by a previous student team, a migration made necessary by the deprecation of ROS 1 and the transition of the school's computers to Ubuntu 24.04.

My internship was structured around two objectives. The first was to verify that this migration was functional, by testing the simulation with complete production scenarios rather than simple startup checks. This approach allowed me to identify that production durations defined in configuration files needed to be adjusted, as the ROS 2 simulation runs faster than its ROS 1 counterpart.

The second objective, the main contribution of this internship, was to extend the simulation to allow product evacuation from any workstation in the cell, whereas only station 3 previously allowed it. This evolution required understanding and modifying the entire relevant software chain: the command interface used by students, the reception and processing of the evacuation request on the robot side, the recording of this information in the simulation log, and finally the automatic conformity verification performed by the module's grading tool. Each component was tested in isolation before integration, with constant attention paid to backward compatibility with existing student code.

Keywords: ROS 2, software migration, Petri net, robotic simulation, flexible workshop, CoppeliaSim

Résumé

Abstract

Introduction

1. Environnement du stage

- 1.1. Les structures d'accueil : LAAS-CNRS et MFJA
- 1.2. Le support de stage : Le TER « Atelier Flexible »
- 1.3. L'architecture logicielle et ROS
- 1.4. Méthodologie de suivi et organisation du travail

2. Problématique du stage

3. Étude de l'existant

- 3.1 Fonctionnement général de la simulation
- 3.2 Organisation du code

4. Objectif 1 : Réalisation de tests pour la vérification de la migration

- 4.1 Démarche
- 4.2 Les Tests
- 4.3 Bilan de l'objectif 1

5. Objectif 2 : Évacuation par un autre poste

- 5.1 Le problème de départ
- 5.2 Comprendre la chaîne d'évacuation existante
- 5.3 La correspondance poste / robot / position
- 5.4 Les choix de conception
- 5.5 Mise en œuvre
- 5.6 Enregistrement du poste d'évacuation dans le log
- 5.7 Vérification du poste d'évacuation par le checker

6. Résultats, analyse et discussion

- 6.1 Résultats de l'objectif 1 : vérification de la migration
- 6.2 Résultats de l'objectif 2 : évacuation par un autre poste

7. Conclusion et bilans

Bilan synthétique des compétences :

Bibliographie / Webographie

ANNEXE

Usage de l'intelligence artificielle générative

Introduction

Ce rapport présente les travaux réalisés dans le cadre de mon stage de fin de quatrième année du cursus Systèmes Robotiques et Interactifs (SRI) de l'école d'ingénieurs UPSSITECH. Ce stage, d'une durée de 12 semaines (du 8 juin au 31 août 2026), s'est déroulé au sein du Laboratoire d'Analyse et d'Architecture des Systèmes (LAAS-CNRS) à Toulouse. Ma mission principale a consisté à valider et faire évoluer l'architecture logicielle d'une simulation pédagogique de robotique industrielle, servant de support de cours pour les étudiants de l'ENSEEIH.

Ce rapport s'organise de la manière suivante : je commence par présenter l'environnement dans lequel s'est déroulé le stage, puis la problématique qui a motivé les travaux. Je détaille ensuite l'existant sur lequel je me suis appuyé, avant de présenter successivement les deux objectifs du stage : la vérification de la migration vers ROS 2, puis l'implémentation de l'évacuation par un autre poste. Je termine par une présentation des résultats obtenus et un bilan de cette expérience.

1. Environnement du stage

1.1. Les structures d'accueil : LAAS-CNRS et MFJA

Ce stage s'est déroulé au sein de l'équipe ROC (Recherche Opérationnelle, Optimisation Combinatoire) du LAAS-CNRS, une équipe spécialisée dans la résolution de problèmes complexes d'ordonnancement et de planification. Mes travaux se sont inscrits dans le cadre des activités de la Maison de la Formation Jacqueline Auriol (MFJA), une plateforme d'enseignement mutualisée toulousaine qui abrite des équipements industriels de pointe. Mon travail a été supervisé par Mme Sandra Ulrich Ngueveu, Docteure au LAAS-CNRS, enseignante à l'ENSEEIH et auteure du projet sur lequel j'ai travaillé, ainsi que par Mme Diane Bury, Docteure et ingénieure en robotique, gestionnaire des plateformes de robotique industrielle et robotique mobile à la MFJA.

1.2. Le support de stage : Le TER « Atelier Flexible »

Le projet sur lequel j'ai travaillé est un TER (Thème d'Étude et de Recherche) intitulé « Atelier Flexible ». Il s'agit d'un module d'enseignement destiné aux étudiants ingénieurs de 3ème année de l'ENSEEIH, dont l'objectif est d'apprendre à commander et optimiser une cellule de production automatisée.

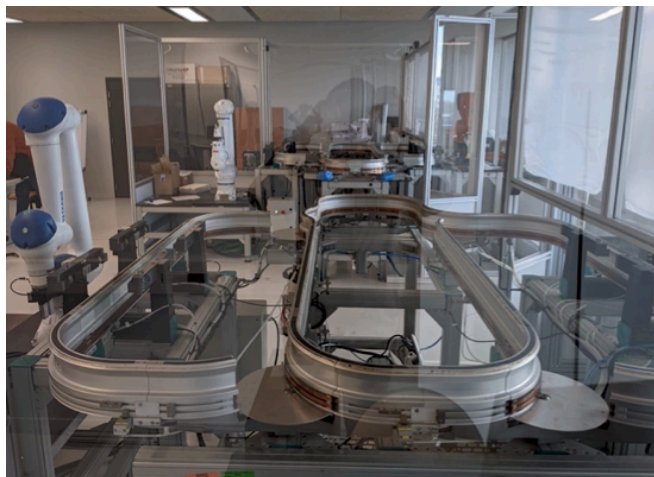


Figure 1. Photographie de la cellule flexible

Concrètement, la cellule est composée de robots, de postes de travail et de navettes circulant sur des rails, qui transportent les produits d'un poste à l'autre. Chaque produit doit suivre une gamme de fabrication : une suite ordonnée de tâches à réaliser sur des postes précis, avec une durée définie pour chacune.

Pour valider ce module, les étudiants doivent modéliser un Réseau de Petri capable de piloter l'atelier. La contrainte majeure de ce TER est la généricité : la solution proposée par les étudiants ne doit pas être codée spécifiquement pour une seule suite de tâches. Le Réseau de Petri doit être capable de s'adapter dynamiquement à n'importe quelle gamme de fabrication donnée en entrée.

1.3. L'architecture logicielle et ROS

La cellule de production étudiée existe physiquement à la MFJA (il s'agit d'une maquette réelle). Pour des raisons pédagogiques, elle possède un « jumeau numérique » simulé. Cette simulation repose sur deux briques logicielles qui communiquent entre elles :

- CoppeliaSim : Le logiciel qui simule la physique de la cellule (la circulation des navettes sur les rails, les aiguillages, les bras robotisés et les postes de travail).
- ROS (Robot Operating System) : Un middleware robotique qui fait le lien entre les différents programmes (appelés nœuds) via un système de communication par *Topics*. Il offre aux étudiants une interface simplifiée pour écrire leur programme de commande.

1.4. Méthodologie de suivi et organisation du travail

Mon avancement a été suivi via une réunion hebdomadaire avec chacune de mes encadrantes, Sandra Ulrich Ngueveu et Diane Bury, l'occasion de présenter mon travail et de fixer les objectifs de la semaine suivante. J'ai complété ce suivi par un journal de bord quotidien, avec un bilan et un objectif définis pour chaque semaine, ainsi qu'un tableau Trello pour organiser mes tâches, particulièrement utile pour découper l'objectif d'évacuation par un autre poste en étapes successives. J'ai également versionné mon travail avec Git, en créant une branche dédiée à chaque fonctionnalité, ce qui m'a permis d'avancer par étapes et de garder une trace de chaque modification apportée au projet.

2. Problématique du stage

L'architecture d'origine du TER était basée sur la première version de ROS (ROS 1). Cependant, ROS 1 est officiellement devenu obsolète le 31 mai 2025 et n'est plus supporté par les systèmes d'exploitation récents, tels qu'Ubuntu 24.04, désormais déployés sur les ordinateurs de l'école. Pour pallier ce problème, un précédent binôme d'étudiants a réalisé la migration complète du projet vers la nouvelle génération du middleware : ROS 2. Mon stage a débuté à la suite de ces travaux de migration et s'articule autour de deux objectifs :

1. Validation de la migration : Mettre à l'épreuve la nouvelle architecture ROS 2 pour s'assurer que la simulation se comporte de manière identique à l'ancienne version, et que les étudiants puissent y exécuter leurs réseaux de Petri sans dysfonctionnement.
2. Évolution fonctionnelle de la cellule : Implémenter une nouvelle fonctionnalité. Jusqu'à présent, les produits manufacturés ne pouvaient être évacués de la chaîne que par un seul poste de travail (le poste 3). Mon objectif est de modifier le code source pour permettre l'évacuation d'un produit depuis n'importe quel poste de la cellule.

3. Étude de l'existant

3.1 Fonctionnement général de la simulation

La version simulée de la cellule est composée de 4 robots, 8 postes de travail, 12 aiguillages, et de navettes qui circulent en sens unique sur des rails, des capteurs de navettes et des actionneurs stop et start. Des produits apparaissent sur un poste puis doivent passer par une suite de tâches (définie dans un fichier de configuration), et finissent par être évacués. Chaque produit et chaque tâche sont matérialisés dans CoppeliaSim par des empilements de cubes colorés.

Côté logiciel, l'étudiant n'a pas besoin d'utiliser de commandes ROS. Il écrit son réseau de Petri dans un fichier `main_commande.cpp`, en utilisant des fonctions simples mises à sa disposition, réparties en trois catégories. Les commandes d'actionneurs, comme `aiguillage.Gauche()` ou `cmd.Stop_PS()`, correspondent directement à des ordres envoyés au système physique (orienter un aiguillage, arrêter une navette). Les commandes liées aux robots, comme `robot.DeplacerPiece()` ou `robot.FaireTache()`, offrent une abstraction de la tâche à effectuer : l'étudiant demande une action de haut niveau sans se soucier de la séquence de mouvements bas niveau qu'elle déclenche. Enfin, les commandes liées aux objets, comme `robot.AjouterProduit()` ou `robot.Evacuer()`, permettent de faire apparaître ou sortir un produit de la simulation. Toute la complexité de la communication ROS (les topics, les messages, les services) est cachée derrière ces fonctions par des classes d'interface. C'est un point que j'ai trouvé bien pensé : la migration de ROS1 vers ROS2 n'a pas changé ce principe, donc un étudiant peut reprendre son ancien code presque tel quel.

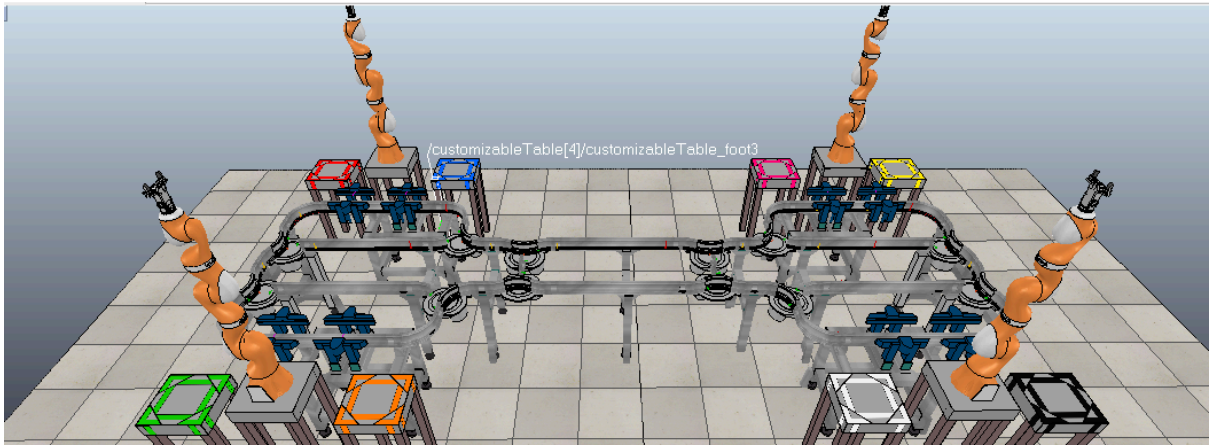


Figure 2. Image de la cellule flexible en simulation

3.2 Organisation du code

Le projet est séparé en deux grands dossiers. Le dossier “**celluleflexible**” contient toute la partie « bas niveau » : les packages ROS (**aiguillages**, **commande**, **commande_locale**, **robots**, **shuttles**, **launcher**), les scripts Lua de CoppeliaSim, et le plugin qui fait le pont entre ROS2 et CoppeliaSim. Le dossier “**etu**” contient tout ce qui concerne l’étudiant : son fichier de commande, les fichiers de configuration des produits, et une série de scripts pour compiler puis lancer puis valider et documenter son travail. Pour donner une idée de l’ampleur du projet, le dossier **celluleflexible** représente à lui seul 3867 fichiers, contre 139 fichiers pour le dossier **etu**, un écart qui reflète bien la séparation voulue entre la complexité bas niveau, réservée aux développeurs, et l’interface simplifiée mise à disposition des étudiants.

J’ai passé mes premiers jours à comprendre cette organisation, car pour tester quoi que ce soit, il fallait d’abord savoir quels fichiers servaient à quoi, et dans quel ordre les choses se lançaient.

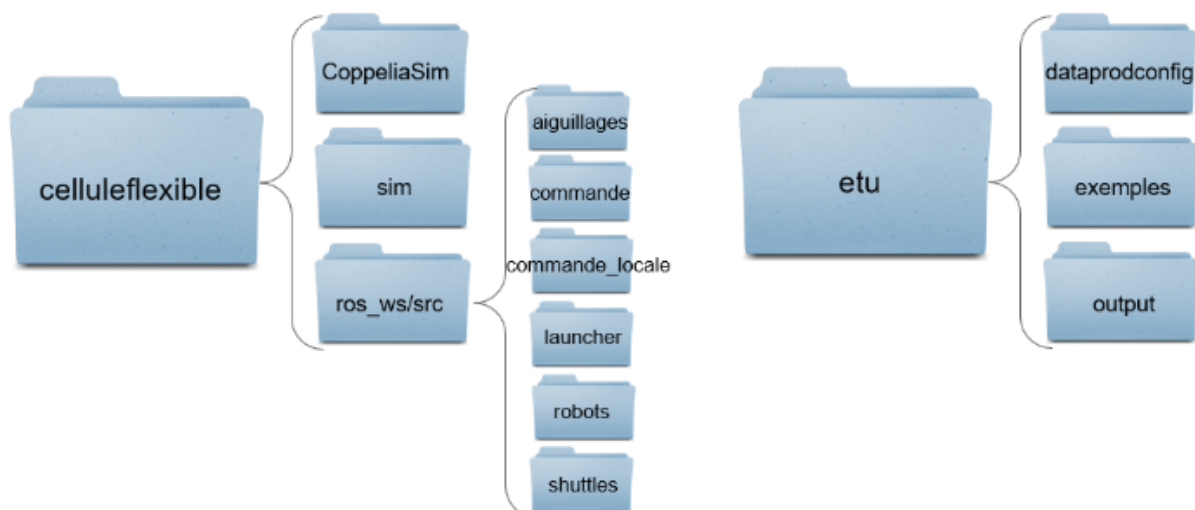


Figure 3. Arborescence du projet

4. Objectif 1 : Réalisation de tests pour la vérification de la migration

4.1 Démarche

Pour travailler pendant ce stage j'ai reçu un ordinateur avec la distribution Ubuntu 24.04. J'ai installé une machine virtuelle sous Ubuntu 20 pour pouvoir faire tourner l'ancienne version ROS1 en parallèle. L'idée était de pouvoir comparer : si un comportement me semblait bizarre en ROS2, je pouvais vérifier comment cela se passait en ROS1 pour savoir si c'était un problème de migration ou un comportement normal.

Le fait que la simulation se lance et que les navettes tournent ne prouve pas que la migration a été réussie car cela ne permet pas de tester toutes les fonctionnalités du projet. Pour vraiment vérifier si la migration était correcte, il fallait mettre le système à l'épreuve avec des scénarios de production complets, qui font intervenir les aiguillages, les arrêts de navette, les transferts de pièces par les robots, les tâches de fabrication et l'évacuation.

J'ai donc choisi de procéder par tests successifs en réutilisant des codes déjà existants puis en écrivant les miens.

4.2 Les Tests

Le premier test a porté sur [Tuto_Basique](#), un exemple fourni avec le projet qui montre une séquence de production simple. Il s'agit du code tutoriel donné aux étudiants, il réalise une production simple avec des données fixées . C'est le premier code que tous les étudiants doivent lancer pour découvrir la simulation.

En le lançant avec la simulation en ROS2, j'ai rencontré un problème : la navette avançait bien, mais à un moment un aiguillage tournait exactement au moment où la navette passait dessus, ce qui provoquait une chute de la navette. En ROS1 sur ma machine virtuelle, le même code fonctionnait parfaitement.

Ma responsable technique m'avait orienté sur une piste : ROS2 sous Ubuntu 24.04 a des temps d'exécution beaucoup plus rapides, donc les temps de productions ne sont plus les mêmes. J'ai d'abord essayé d'ajouter une temporisation, mais ça ne réglait rien. En poursuivant l'analyse, j'ai compris que le vrai problème venait des durées de fabrication : comme la simulation ROS2 va plus vite, une tâche configurée pour durer 13 secondes se terminait bien plus tôt, la navette repartait trop tôt et rattrapait l'aiguillage avant qu'il ait fini de se positionner. Or [Tuto_Basique](#) ne vérifie à aucun moment que l'aiguillage a bien terminé son positionnement avant le passage de la navette.

Comme les durées ne sont plus les mêmes j'ai mis à jour le fichier d'entrée (gamme). En effet ce code de tutoriel ne marche pas pour n'importe quelle gamme de fabrication, ce qui est voulu. Il faut montrer aux étudiants qu'un code non robuste ne fonctionne pas toujours.

La solution a donc été d'ajuster la durée de la tâche dans le fichier de configuration [dataP2_1_1_n0](#). Après quelques essais, j'ai trouvé qu'avec une durée de 3 secondes au lieu de 13, le code [Tuto_Basique](#) refonctionnait correctement.

Test d'un code étudiant récent et fonctionnel

Pour aller plus loin que les exemples fournis, j'ai récupéré un code écrit par un ancien binôme d'étudiants (DU_NO) que j'ai adapté au formalisme ROS2. C'était un code plus robuste, qui répond au cahier des charges pour plusieurs gammes de fabrication. L'objectif était de vérifier qu'un code étudiant « réel », plus complexe que les tutoriels, fonctionnait bien après la migration.

Le résultat a été positif : le code fonctionnait, et surtout la migration n'impose pas de changer la manière de coder le réseau de Petri. Un étudiant qui avait écrit son code en ROS1 pouvait le reprendre en ROS2 sans revoir sa logique.

Écriture de mes propres codes de test

Après avoir validé des codes existants, j'ai écrit mes propres réseaux de Petri pour finir de tester la migration avec des scénarios que je maîtrisais entièrement. J'ai commencé par des tests très ciblés (par exemple faire circuler une navette jusqu'à un poste, y créer un produit, puis l'acheminer jusqu'au poste 3 pour l'évacuer), puis j'ai construit un réseau de Petri personnel plus complet.

Écrire mes propres codes m'a obligé à comprendre en profondeur comment fonctionnent les fonctions mises à disposition de l'étudiant : comment on arrête une navette à un point précis avec `cmd.Stop_PS()`, comment on vérifie qu'elle est bien arrivée avec `capteur.get_PS()`, comment on enchaîne les transitions sans jamais bloquer la boucle principale. Mon code personnel a fini par fonctionner correctement, ce qui a clôturé cette phase de tests.

4.3 Bilan de l'objectif 1

Au terme de cette série de tests j'ai pu conclure que la migration vers ROS2 est fonctionnelle pour l'usage qu'en feront les étudiants. Le seul point d'attention identifié concerne les durées de fabrication, qui doivent être réajustées dans les fichiers de configuration pour tenir compte de la vitesse d'exécution plus élevée de ROS2, qui peut faire échouer les codes les moins robustes en raison des temps d'exécutions qui ne sont pas les mêmes qu'en ROS1.

5. Objectif 2 : Évacuation par un autre poste

5.1 Le problème de départ

Dans l'état actuel du projet, l'évacuation d'un produit ne peut se faire que par le poste 3. Quand l'étudiant écrit `robot.Evacuer()` dans son réseau de Petri, le produit posé sur le poste 3 disparaît de la simulation. C'est la seule sortie possible. L'objectif de ma contribution principale est de lever cette limitation, pour qu'un produit puisse être évacué depuis n'importe quel poste, choisi par l'étudiant ou imposé par l'enseignant.

À terme, l'idée est même d'aller plus loin : le fichier de configuration définirait quels postes sont autorisés à évacuer, et l'étudiant choisirait parmi ces postes autorisés celui qu'il veut utiliser, toujours via un paramètre passé à la fonction `Evacuer`. Pour une première version, je me suis concentré sur la mécanique de base : rendre l'évacuation possible depuis n'importe quel poste passé en paramètre. Puis je me suis dirigé vers le fonctionnement incluant les postes autorisés.

5.2 Comprendre la chaîne d'évacuation existante

Avant de modifier quoi que ce soit, j'ai voulu comprendre exactement comment une demande d'évacuation circulait à travers les différents programmes. J'ai retracé cette chaîne fichier par fichier.

Quand l'étudiant appelle `robot.Evacuer()`, l'appel passe d'abord par une classe d'interface, `RobotsInterface`, dont le rôle est de traduire cette instruction simple en un message ROS. Dans la version d'origine, cette fonction publiait un message vide sur un topic nommé `/commande/Simulation/Evacuer`. Ce message vide servait uniquement de signal « évacue », sans transporter aucune information.

Ce topic est écouté par les 4 robots. Un point que j'ai mis un certain temps à comprendre est qu'au lancement de la simulation, le programme `robot` est lancé quatre fois séparément, chaque fois avec un numéro différent passé en argument. Chaque objet `robot` garde ce numéro dans une variable, et c'est ce qui lui permet de savoir qui il est. J'ai vérifié cela dans les fichiers de lancement, où l'on voit bien quatre appels distincts au même exécutable avec les arguments 1, 2, 3 et 4.

Quand le message d'évacuation arrive, les 4 instances robot traitent le message séparément, mais la fonction `Robot::Evacuer` ne faisait réellement quelque chose que pour le robot 2, à cause d'un test écrit en dur : `if(num_robot==2)`. À l'intérieur, la position était également figée sur le poste 3. C'est cette double contrainte en dur qu'il fallait remplacer par un système dynamique.

5.3 La correspondance poste / robot / position

La documentation indique que chaque robot dessert deux postes, repérés en interne par les positions 1 et 4 (les positions 2 et 3 correspondant à des positions de navette, pas de poste). Je me suis rendu compte qu'il existait déjà une fonction, `computeTableId`, qui fait déjà la correspondance entre un couple (robot, position) et l'emplacement physique du poste dans CoppeliaSim, et qui gère déjà les 4 robots. C'était une bonne nouvelle : la partie « faire disparaître les cubes au bon endroit » fonctionnait déjà pour tous les robots, je n'avais donc pas à la réécrire.

À partir de là, j'ai pu établir la correspondance complète entre les 8 postes et les couples (robot, position), résumée dans le tableau suivant :

Robot	Position	Poste
Robot 1	4	Poste 1
Robot 1	1	Poste 2
Robot 2	1	Poste 3
Robot 2	4	Poste 4
Robot 3	1	Poste 5
Robot 3	4	Poste 6
Robot 4	1	Poste 7
Robot 4	4	Poste 8

Tableau 1. Correspondance des postes

5.4 Les choix de conception

Plusieurs décisions ont dû être prises : où faire la traduction poste vers robot + position ? Deux options étaient possibles : soit calculer le robot cible côté commande centrale et n'envoyer le message qu'à lui, soit diffuser le numéro de poste à tous les robots et laisser chaque robot vérifier lui-même si le poste le concerne. J'ai choisi la deuxième option, parce que c'était déjà comme ça que fonctionne le reste du projet : pour les autres actions (déplacer une pièce, faire une tâche), la commande est diffusée à tous les robots et chacun filtre selon son numéro. Rester cohérent avec l'architecture existante me semblait plus correct.

Comment écrire la correspondance ? J'ai choisi de l'écrire de la manière la plus explicite possible, avec une série de huit tests, un par poste, chacun disant clairement à quel robot et à quelle position correspond le poste. Une version plus condensée aurait été possible, mais j'ai préféré privilégier la lisibilité : le projet est repris chaque année par de nouveaux étudiants et un code clair est plus facile à comprendre et à corriger.

Quel type de message choisir ? Le message d'origine était vide. Pour transporter le numéro de poste, j'ai choisi de créer mon propre type de message contenant un seul entier, plutôt que de réutiliser un type standard. L'avantage est la modularité : si plus tard on veut transporter d'autres informations (par exemple la liste des postes autorisés), il suffira d'ajouter un champ à ce message sans changer le type du topic partout.

Compatibilité avec les anciens codes ? Changer la fonction `Evacuer` pour qu'elle prenne un paramètre risquait de casser tous les codes étudiants existants qui appellent `robot.Evacuer()` sans argument. Pour éviter ça, j'ai utilisé une valeur par défaut : la fonction s'appelle désormais `Evacuer(int numero_poste = 3)`.

Ainsi, un ancien code qui écrit `robot.Evacuer()` continue de fonctionner exactement comme avant (évacuation par le poste 3), tandis qu'un nouveau code peut préciser un autre poste. Cela m'a permis d'ajouter la fonctionnalité sans rien casser.

5.5 Mise en œuvre

L'implémentation touche deux packages : `commande`, qui contient l'interface utilisée par l'étudiant et qui se charge d'envoyer la demande d'évacuation, et `robots`, qui reçoit cette demande et exécute réellement l'évacuation.

Un point mérite d'être clarifié pour éviter toute confusion. Le nouveau type de message que j'ai créé pour transporter le numéro de poste est défini dans le package `robots`. Ce choix n'a rien à voir avec le sens de circulation du message : je l'ai placé là parce que le package `robots` disposait déjà de toute l'infrastructure nécessaire pour définir des messages (le dossier dédié et la configuration de génération), alors que le package `commande` n'en avait pas. Définir le message ailleurs aurait demandé d'ajouter cette infrastructure de zéro dans `commande`, ce qui était plus lourd et introduisait une nouvelle dépendance. Une fois défini dans `robots`, ce message peut être utilisé aussi bien par le package qui l'envoie que par celui qui le reçoit.

Concrètement, la circulation est donc la suivante : le message est défini dans `robots`, il est envoyé depuis `commande` (par la classe d'interface `RobotsInterface`), et il est reçu dans `robots` (par la classe `Robot`).

Côté définition, j'ai créé le fichier décrivant le nouveau message, qui ne contient qu'un seul champ : le numéro de poste. Je l'ai ensuite déclaré dans le fichier de configuration de compilation du package `robots` pour qu'il soit généré. Avant même de l'utiliser dans le code, j'ai vérifié en ligne de commande que le système le reconnaissait correctement, ce qui m'a permis de valider cette première brique isolément.

Côté envoi, dans la classe `RobotsInterface` du package `commande`, j'ai modifié la fonction `Evacuer` pour qu'elle accepte désormais un numéro de poste en paramètre, et qu'elle place ce numéro dans le message publié, au lieu d'envoyer un message vide comme auparavant. C'est également ici que j'ai ajouté la valeur par défaut permettant de conserver la compatibilité avec les anciens codes.

Côté réception, dans la classe `Robot` du package `robots`, j'ai adapté l'écoute pour qu'elle corresponde au nouveau type de message, puis j'ai réécrit la fonction d'évacuation. Cette fonction lit maintenant le numéro de poste reçu, détermine grâce à une série de tests quel robot et quelle position sont concernés, et ne déclenche l'évacuation que si le robot courant est bien celui qui dessert ce poste. Toute la logique interne existante (la lecture des couleurs des cubes puis leur disparition) a été conservée telle quelle, en l'adaptant simplement pour qu'elle utilise la position déterminée dynamiquement au lieu d'être figée sur le poste 3.

5.6 Enregistrement du poste d'évacuation dans le log

Une fois l'évacuation par un autre poste fonctionnelle, il restait à en garder une trace exploitable. En effet, permettre d'évacuer par n'importe quel poste n'a de sens que si l'on peut ensuite vérifier que l'étudiant a bien évacué par un poste autorisé. Pour cela, il fallait que l'information « évacuation par le poste X » soit enregistrée quelque part.

Avant de coder quoi que ce soit, j'ai voulu comprendre précisément ce qu'était le « log » du projet, car je n'en avais qu'une idée vague. En explorant le fichier `LogManager.cpp`, j'ai compris qu'il ne s'agissait pas d'un simple affichage à l'écran, mais d'un véritable fichier texte (`log.txt`) dans lequel chaque événement de la simulation vient écrire une ligne : l'apparition d'un produit, la fin d'une tâche, une évacuation, ou encore une erreur. Chaque ligne constate simplement un fait, sans porter de jugement dessus.

Cette lecture a été importante, car elle a clarifié une question d'architecture sur laquelle j'ai hésité. Au départ, je pensais qu'il fallait vérifier directement, au moment de l'évacuation, si le poste était autorisé, et écrire dans le log si ce n'était pas le cas. Mais en observant comment les autres événements sont traités, j'ai constaté que le système sépare volontairement deux rôles : le `LogManager` se contente de constater les faits et de les écrire, tandis que la vérification et le jugement (« est-ce conforme à ce qui était attendu ? ») sont entièrement centralisés dans un outil séparé, le checker, qui relit le log après coup. J'ai donc choisi de respecter cette séparation plutôt que de mélanger les deux rôles : le log devait uniquement enregistrer par quel poste l'évacuation avait eu lieu, et c'est au checker que reviendrait plus tard la tâche de comparer ce poste avec la liste des postes autorisés.

J'ai donc regardé comment l'information circulait de la fonction d'évacuation jusqu'au log. J'ai constaté que la fonction `Robot::Evacuer` envoyait déjà au `LogManager`, à chaque évacuation, un tableau d'entiers contenant les couleurs des cubes du produit. Plutôt que de créer un nouveau canal de communication, j'ai choisi de réutiliser ce tableau existant en y ajoutant simplement le numéro de poste en première position. Ce choix m'a paru le plus simple et le moins risqué : il évitait de créer un nouveau type de message et ne touchait qu'à une seule ligne côté envoi.

Côté réception, dans le `LogManager`, j'ai adapté la fonction qui traite les évacuations pour qu'elle lise ce numéro de poste en tête du tableau et l'écrive dans la ligne de log. Le seul point délicat était le décalage des indices : puisque le poste occupe désormais la première case du tableau, les couleurs se retrouvaient décalées d'une position, et il fallait en tenir compte à la lecture pour ne pas se tromper de valeur. C'est pour cette raison que j'ai modifié les deux fichiers (l'envoi et la réception) en même temps, puis testé immédiatement pour vérifier que rien n'était décalé.

Le test a confirmé que la modification fonctionnait : après une évacuation par le poste 3, la ligne correspondante du fichier de log faisait bien apparaître le numéro de poste au bon endroit, et les couleurs restaient correctes. Le log contient désormais toute l'information nécessaire pour que le checker puisse vérifier la conformité des évacuations.

5.7 Vérification du poste d'évacuation par le checker

La dernière étape pour rendre l'évacuation par un autre poste réellement exploitable concernait le checker. En effet, permettre d'évacuer par n'importe quel poste et enregistrer ce poste dans le log ne sert à rien si personne ne vérifie ensuite que l'étudiant a bien respecté les postes qui lui étaient autorisés. C'est précisément le rôle du checker.

Avant de le modifier, j'ai dû comprendre en profondeur ce qu'était cet outil. Le checker est un programme écrit en Python, totalement indépendant de la simulation : il ne communique avec aucun autre programme pendant la simulation. On le lance à la main, une fois la simulation terminée, en lui donnant deux fichiers en entrée : le fichier de configuration des produits (celui qui décrit ce que l'étudiant était censé produire) et le fichier de log (celui qui décrit ce qui s'est réellement passé pendant la simulation).

Son principe est de comparer ces deux fichiers pour dire si l'étudiant a respecté le cahier des charges. En explorant son code, j'ai vu qu'il fonctionne en quatre grandes étapes : d'abord il lit le fichier de configuration et en extrait les données attendues (types de produits, séquences de tâches, durées, quantités), ensuite il lit le log ligne par ligne, en traitant chaque événement selon son premier mot (apparition d'un produit, tâche effectuée, sortie, etc.) puis il compare ce qui était attendu à ce qui s'est passé, et signale chaque écart par un message d'erreur. Enfin, si tout est correct, il affiche des statistiques sur la production.

J'ai aussi compris que le checker utilise une variable centrale, que je nomme le témoin de validité : dès qu'une erreur de fond est détectée (une mauvaise durée, un mauvais nombre de produits, une tâche sur un poste vide...), cette variable passe à zéro, ce qui signale que la solution de l'étudiant ne respecte pas le cahier des charges. Cette manière de faire m'a servi de modèle pour ma propre vérification.

Mon objectif a été le suivant : faire lire au checker le poste d'évacuation que j'avais ajouté dans le log, et vérifier que ce poste faisait bien partie des postes autorisés définis dans la configuration.

En lisant le code, j'ai d'abord repéré un point important : le checker partait du principe, écrit en dur, que l'évacuation se faisait toujours par le poste 3. Un de ses messages d'erreur mentionnait même explicitement « évacuation vide au poste 3 », avec un commentaire précisant que seul ce poste pouvait évacuer. Cette hypothèse fixe était exactement ce que mon travail devait modifier.

J'ai ensuite remarqué un problème plus subtil mais critique. Comme j'avais ajouté le numéro de poste au début de la ligne « Sortie » du log, tout le reste de la ligne se retrouvait décalé d'une position. Or le checker lisait cette ligne en supposant l'ancien format, sans le poste. Si je ne corrigeais pas cette lecture, le checker allait interpréter mon numéro de poste comme s'il s'agissait du numéro de produit, et se tromper sur toute l'analyse des sorties. C'était le point le plus délicat, car une simple erreur d'indice aurait faussé toute la vérification.

Enfin, j'ai remarqué que le checker vérifiait que le fichier de configuration contenait exactement les quatre variables attendues, ni plus ni moins. Ma nouvelle variable listant les postes autorisés serait donc rejetée si je ne l'ajoutais pas explicitement à la liste des variables acceptées.

À partir de cette analyse, j'ai réalisé plusieurs modifications. J'ai d'abord autorisé la nouvelle variable des postes d'évacuation dans la vérification des clés de configuration, pour que le checker accepte le fichier au lieu de le rejeter. J'ai ensuite ajouté la récupération de cette liste de postes autorisés, en prévoyant le cas où un ancien fichier de configuration ne la contiendrait pas, afin de garder la compatibilité avec l'existant.

J'ai ensuite corrigé la lecture de la ligne « Sortie » pour tenir compte du numéro de poste ajouté au début : j'ai décalé tous les indices de lecture d'une position, de façon à ce que le poste soit lu au bon endroit et que le produit et les tâches soient à nouveau lus correctement.

Une fois le poste correctement lu, j'ai ajouté la vérification qui constitue le cœur de l'objectif : si le poste d'évacuation ne fait pas partie des postes autorisés, le checker affiche un message d'erreur explicite indiquant quel poste a été utilisé et quels postes étaient autorisés, et il fait passer le témoin de validité à zéro. J'ai choisi de traiter cela comme une erreur bloquante, et non comme un simple avertissement, pour rester cohérent avec la philosophie du checker : tout ce qui ne respecte pas la configuration est considéré comme une faute. Évacuer par un poste interdit fait partie du cahier des

charges, il m'a donc semblé logique de le sanctionner au même titre qu'une mauvaise durée ou un mauvais nombre de produits.

Enfin, j'ai adapté le message d'erreur de l'évacuation vide, qui mentionnait le poste 3 en dur, pour qu'il affiche désormais le vrai numéro de poste concerné.

J'ai validé ces modifications en deux temps. J'ai d'abord testé le cas conforme : en configurant les postes 3 et 4 comme autorisés et en évacuant par le poste 3 puis 4, le checker a bien accepté la solution, sans afficher d'erreur d'évacuation, et est allé jusqu'au bout de son analyse. Cela confirmait que la lecture décalée de la ligne « Sortie » fonctionnait, puisque le produit et sa tâche étaient correctement identifiés.

J'ai ensuite testé le cas non conforme : en modifiant le log pour simuler une évacuation par un poste non autorisé, le checker a bien détecté le problème et affiché le message d'erreur attendu, en précisant le poste utilisé et la liste des postes autorisés. Ces deux tests, l'un conforme et l'autre non, m'ont permis de m'assurer que la vérification fonctionnait dans les deux situations, et pas seulement dans le cas favorable, j'ai ensuite testé les 6 autres postes possibles, aussi bien pour le cas conforme que pour le cas non conforme.

6. Résultats, analyse et discussion

6.1 Résultats de l'objectif 1 : vérification de la migration

La méthodologie d'évaluation a consisté à faire tourner successivement plusieurs codes de complexité croissante sur la simulation ROS 2, en comparant leur comportement à celui observé sur la machine virtuelle ROS 1, et en ajustant si besoin les fichiers de configuration associés. Le tableau suivant synthétise les tests réalisés.

Code testé	Fichier de configuration	Durée de tâche initiale	Résultat initial	Durée ajustée	Résultat final
Tuto_Basique	dataP2_1_1_n0	13 s	Échec (collision navette/aiguillage)	3 s	Succès
TeamSALLAG	dataP2_1_1_n0	13 s	Échec (même cause)	3 s	Succès
DU_NO (code étudiant réel)	gamme de fabrication multi-produits	—	—	—	Succès
Code personnel (validation ciblée)	scénario poste 1 vers poste 3, évacuation	—	—	—	Succès

Tableau 2. Synthèse des tests de vérification de la migration ROS 2

Le résultat le plus significatif est l'identification de la cause de l'échec initial des deux premiers codes : il ne s'agissait pas d'un défaut de la migration ROS 2 elle-même, mais d'une conséquence attendue de la différence de vitesse d'exécution entre ROS 1 et ROS 2. Une fois les durées de tâches réajustées, les quatre codes testés ont fonctionné correctement, ce qui valide la migration pour l'usage qu'en feront les étudiants : un réseau de Petri correctement conçu, respectant les principes de robustesse attendus par le TER (ne pas coder une durée en dur sans marge, vérifier l'état des capteurs avant de faire avancer une navette), continue de fonctionner sous ROS 2 sans modification de sa logique.

6.2 Résultats de l'objectif 2 : évacuation par un autre poste

La validation de cette fonctionnalité s'est faite à trois niveaux, correspondant aux trois briques logicielles modifiées : la compatibilité avec les anciens codes, l'évacuation effective sur l'ensemble des postes, et la vérification par le checker.

Compatibilité ascendante. J'ai vérifié qu'un appel `robot.Evacuer()` sans argument continuait à évacuer par le poste 3 comme avant la modification, ainsi qu'un appel explicite `robot.Evacuer(3)`. Les

deux cas ont donné le résultat attendu, confirmant qu'aucun code étudiant existant n'est affecté par cette évolution.

Évacuation multi-poste. J'ai testé l'évacuation successivement sur l'ensemble des huit postes de la cellule, en faisant apparaître un produit sur chaque poste puis en demandant son évacuation par ce même poste. Les huit tests ont été concluants : le produit disparaît systématiquement du poste demandé, ce qui valide la correspondance établie entre chaque poste, le robot qui le dessert et sa position.

Vérification par le checker. J'ai testé le comportement du checker sur l'ensemble des huit postes, à chaque fois dans les deux configurations possibles : le poste testé figurant dans la liste des postes autorisés de la configuration, et le poste testé en étant absent. Dans le premier cas, le checker valide systématiquement la solution sans signaler d'erreur. Dans le second, il détecte la non-conformité, affiche un message précisant le poste utilisé et les postes autorisés, et invalide la solution. Ce test croisé sur les huit postes, dans les deux configurations, m'a permis de m'assurer que la vérification ne se limitait pas à un cas particulier mais fonctionnait de manière générale.

Type de test	Postes testés	Postes autorisés (configuration)	Résultat attendu	Résultat obtenu
Compatibilité, appel sans argument	Poste 3 (implicite)	3, 4	Évacuation réussie	Conforme
Compatibilité, appel avec argument explicite	Poste 3	3, 4	Évacuation réussie	Conforme
Évacuation multi-poste	Postes 1 à 8, un test par poste	—	Disparition du produit sur chaque poste testé	Conforme, 8 tests sur 8
Vérification checker, cas conforme	Postes 1 à 8, un test par poste	Poste testé inclus dans la liste	Validation sans erreur	Conforme, 8 tests sur 8
Vérification checker, cas non conforme	Postes 1 à 8, un test par poste	Poste testé exclu de la liste	Détection et rejet de la solution	Conforme, 8 tests sur 8

Tableau 3. Synthèse des tests de l'évacuation par un autre poste

7. Conclusion et bilans

Bilan technique : Les deux objectifs fixés au début de ce stage ont été atteints avec succès. La campagne de tests a permis de valider la robustesse de la migration vers ROS 2 effectuée précédemment, confirmant que le TER pourra être dispensé sur les nouveaux environnements Ubuntu 24 de l'école sans impacter la logique de résolution demandée aux étudiants. De plus, l'évolution logicielle apportée permet désormais l'évacuation des produits depuis n'importe quel poste de la cellule. Cette flexibilité ouvre la porte à de nouveaux cahiers des charges potentiels pour les enseignants qui utilisent cette simulation de la plateforme et par conséquent des scénarios de production plus complexes pour les futures promotions.

Bilan personnel : D'un point de vue technique, ce stage m'a permis de consolider des compétences en C++ que je n'avais pratiquées jusque-là que dans un cadre scolaire, en les appliquant cette fois sur une base de code réelle, volumineuse et écrite par plusieurs personnes différentes au fil des années. J'ai dû apprendre à naviguer dans un projet que je n'avais pas conçu, à retrouver l'information pertinente au milieu de fichiers nombreux, et à distinguer ce qui relevait de l'architecture générale de ce qui était propre à une fonctionnalité précise. J'ai également découvert en profondeur le fonctionnement du middleware robotique, ROS, à travers ses mécanismes de publication et d'abonnement, ainsi que la manière dont plusieurs processus indépendants peuvent collaborer pour piloter un même système physique.

Organiser mon travail en branches avec Git dédiées à chaque fonctionnalité m'a aussi habitué à découper mes tâches en étapes cohérentes, chacune correspondant à un état du code qui compile et qui a été testé.

Sur le plan méthodologique, j'ai constaté qu'explorer un code inconnu devient beaucoup plus efficace lorsqu'on poursuit un objectif précis d'implémentation : plutôt que de chercher à tout comprendre d'un bloc, le fait de devoir modifier une fonctionnalité précise m'a conduit à cibler mes recherches sur les parties du code réellement concernées, et à en comprendre le fonctionnement par la pratique plutôt que par la seule lecture. J'ai aussi pris l'habitude de tester chaque modification de manière isolée avant de l'intégrer au reste du système, ce qui m'a permis, à plusieurs reprises, de localiser rapidement l'origine d'un problème plutôt que de devoir le chercher dans l'ensemble des changements effectués.

Au-delà des compétences techniques, ce stage m'a surtout donné une vraie expérience concrète de la robotique industrielle et des chaînes de production, un domaine que je n'avais jusque-là abordé que de manière théorique. Manipuler au quotidien les concepts de cellule de production, de gamme de fabrication, de coordination entre plusieurs robots et de contraintes physiques de circulation m'a permis de mieux comprendre les problématiques réelles de ce secteur. Cette expérience a confirmé mon envie de m'orienter vers la robotique industrielle, et plus particulièrement vers les bras manipulateurs, pour la suite de mon parcours. Je considère ce stage comme une expérience professionnelle marquante dans cette direction.

Enfin, le suivi hebdomadaire avec mes deux encadrantes, associé à la tenue d'un journal de bord quotidien avec un objectif défini pour chaque semaine, m'a appris à mieux structurer mon avancement et à formuler plus clairement l'état de mon travail.

Bilan synthétique des compétences :

Développement logiciel : Consolidation du C++ sur une base de code réelle (gestion mémoire par pointeurs partagés, programmation orientée objet) et découverte de la programmation asynchrone, où le code ne doit jamais bloquer une boucle d'exécution en attendant un événement.

Robotique : Utilisation approfondie de ROS 2 : création de nœuds, définition de messages personnalisés, communication par publication et abonnement entre processus indépendants. Compréhension du fonctionnement d'une cellule de production automatisée et des réseaux de Petri comme outil de commande.

Méthodologie : Décomposition d'un problème en étapes testables indépendamment, validation de chaque brique avant intégration, gestion de version avec Git par commits atomiques. Exploration efficace d'une base de code inconnue en gardant un objectif d'implémentation précis.

Compétences transversales : Structuration et communication de l'avancement via un suivi hebdomadaire et un journal de bord.

Perspectives : Maintenant que le jumeau numérique sous ROS 2 est stable et enrichi, la perspective logique pour ce projet est le pont avec la maquette réelle de la MFJA. Le travail futur consistera à lier les commandes ROS 2 développées dans le simulateur CoppeliaSim aux automates réels, afin que le code des étudiants puisse piloter l'atelier physique avec le même niveau de flexibilité.

En parallèle de mes deux objectifs principaux, j'ai également commencé à travailler sur une piste d'amélioration identifiée par le binôme précédent dans son rapport de migration : permettre à l'utilisateur de choisir librement quelles navettes faire apparaître en début de simulation, plutôt que de se voir imposer les premières navettes disponibles dans un ordre fixe. J'ai localisé la fonction responsable du chargement des navettes dans CoppeliaSim, qui accepte déjà en paramètre le numéro de la navette à charger, ce qui rend cette évolution réalisable sans modification lourde de l'architecture existante. Le travail restant consiste à ajouter une interaction avec l'utilisateur pour recueillir son choix et à adapter l'appel à cette fonction en conséquence.

Bibliographie / Webographie

La documentation ROS2 Jazzy :

<https://docs.ros.org/en/jazzy/How-To-Guides/Migrating-from-ROS1.html>

<https://docs.ros.org/en/jazzy/How-To-Guides/Developing-a-ROS-2-Package.html>

<https://docs.ros.org/en/jazzy/Installation.html>

<https://docs.ros.org/en/jazzy/How-To-Guides/Launching-composable-nodes.html>

La documentation officielle CoppeliaSim :

<https://www.coppeliarobotics.com/helpFiles/>

Le cours complet ROS et ROS2 de Olivier Stasse :

<https://homepages.laas.fr/ostasse/Teaching/ROS/poly-rosintro.pdf>

Quelques vidéos explicatives :

<https://www.youtube.com/watch?v=WhX9-wCiFYE>

<https://www.youtube.com/watch?v=Yucc8UTwcw0>

<https://www.youtube.com/watch?v=mjrx8EFSb8>

<https://www.youtube.com/watch?v=EJfTeRoW4cQ>

Les dépôts Github du projet :

<https://github.com/ngueveu/shuttles-and-robots-challenge-for-students>

https://github.com/aip-primeca-occitanie/cellule_flexible_coppelia-2025-2026

Les documents internes :

Poly de présentation du TER Atelier Flexible (S.U. Ngueveu, S. Caux), ENSEEIHT, année 2025/2026

Rapport du projet long ACISE 2025/2026 « Migration en ROS2 du TER Atelier flexible », E. Norroy et E. Dubois

ANNEXE

Usage de l'intelligence artificielle générative

Conformément à la charte de l'Université de Toulouse sur l'utilisation des IA génératives, je précise ici la nature de l'usage que j'en ai fait pour la rédaction de ce rapport.

J'ai utilisé une IA générative comme outil d'aide à la rédaction, à partir de mes propres notes et de mon expérience de stage. Cet usage a porté sur :

- La reformulation et la structuration de certains paragraphes
- L'aide à la relecture pour repérer des formulations imprécises ou trop informelles